MAJOR PROJECT REPORT
ON

Ledger: AI-Based Fake News Detection System Using Explainable AI

Submitted By

ANANDHAKRISHNAN S (CEC23CS033)
ARAVIND A KAMATH (CEC23CS041)
ARJUN MANOJ (CEC23CS043)
SANGEETH KRISHNAN S (CEC23CS105)

under the esteemed guidance of

Mrs. Rajeswary R
Assistant Professor
Department of Computer Science and Engineering



SEPTEMBER 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING, PALLIPPURAM P O, CHERTHALA,
ALAPPUZHA PIN: 688541,
PHONE: 0478 2553416, FAX: 0478 2552714
http://www.cectl.ac.in

MAJOR PROJECT REPORT ON

Ledger: AI-Based Fake News Detection System Using Explainable AI

Submitted By

ANANDHAKRISHNAN S (CEC23CS033)
ARAVIND A KAMATH (CEC23CS041)
ARJUN MANOJ (CEC23CS043)
SANGEETH KRISHNAN S (CEC23CS105)

under the esteemed guidance of

Mrs. Rajeswary R

In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

of

APJ Abdul Kalam Technological University



SEPTEMBER 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING, PALLIPPURAM P O, CHERTHALA,
ALAPPUZHA PIN: 688541,
PHONE: 0478 2553416, FAX: 0478 2552714
http://www.cectl.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COLLEGE OF ENGINEERING CHERTHALA

ALAPPUZHA-688541

C E R T I F I C A T E

This is to certify that, the project report titled **LEDGER: AI-BASED FAKE NEWS DETECTION SYSTEM USING EXPLAINABLE AI**, is a bonafide record of the CSD 415 Project Phase I presented by **ANANDHAKRISHNAN S** (Reg. No. CEC23CS033), **ARAVIND A KAMATH** (Reg. No. CEC23CS041), **ARJUN MANOJ** (Reg. No. CEC23CS043), **SANGEETH KRISHNAN S** (Reg. No. CEC23CS105), Seventh Semester B. Tech. Computer Science and Engineering students, under our guidance and supervision, in partial fulfillment of the requirements for the award of the degree, B. Tech. Computer Science and Engineering of APJ Abdul Kalam Technological University.

Guide	**Co-ordinator**	**HOD**
Mrs. Rajeswary R	**Mrs. Rakhi R**	**Dr. Preetha Theresa Joy**
Assistant Professor	Assistant Professor	Professor
Dept. of Computer Science & Engineering	Dept. of Computer Science & Engineering	Dept. of Computer Science & Engineering

ACKNOWLEDGEMENT

This work would not have been possible without the support of many people. First and foremost, we give thanks to Almighty God who gave us the inner strength, resource and ability to complete our project successfully.

We would like to thank **Dr. Jaya V.L**, our Principal, who has provided us with the best facilities and atmosphere for the project completion and presentation. We would also like to thank our HOD **Dr. Preetha Theresa Joy**, Professor, Department of Computer Science and Engineering, our Project Co-ordinators **Mrs. Rakhi R**, Assistant Professor, Department of Computer Science and Engineering, **Dr. Shyama Das**, Professor, Department of Computer Science and Engineering and our guide **Mrs. Rajeswary R**, Assistant Professor, Department of Computer Science and Engineering for the help extended and also for the encouragement and support given to us while doing the project.

We would like to thank our dear friends for extending their cooperation and encouragement throughout the project work, without which we would never have completed the project this well. Thank you all for your love and also for being very understanding.

ABSTRACT

The rapid proliferation of fake news and misinformation across social media and digital platforms has emerged as a critical societal challenge, distorting democratic elections, propagating harmful medical myths and inciting communal unrest. While existing automated detection systems achieve high predictive accuracy, they predominantly operate as opaque black boxes that yield classifications without explaining the underlying reasoning, thereby undermining user trust and institutional credibility. To address these critical limitations, this project proposes **Ledger**, an Explainable AI-assisted web application for real-time fake news verification. The core classification architecture integrates **FastText** subword embeddings to robustly capture semantic representations while effectively handling out-of-vocabulary terms, informal internet slang and typographical errors. A **1D Convolutional Neural Network (CNN)** extracts salient local n-gram features and deceptive syntactic patterns, which are sequentially processed by a **Long Short-Term Memory (LSTM)** network to model long-range contextual dependencies across the text. To overcome black-box opacity, the system incorporates **LIME** (Local Interpretable Model-agnostic Explanations) to generate word-level attribution heatmaps that visually explain individual veracity decisions alongside calibrated confidence scores. Grounded in the methodology of the base paper, *Advancing Fake News Detection: Hybrid Deep Learning With FastText and Explainable AI* (Hashmi et al., IEEE Access, 2024), the proposed implementation pairs a high-performance Python **FastAPI** backend for sub-second inference with an interactive **React.js** web dashboard. This Phase I report details the problem formulation, a comprehensive literature survey, the proposed system, feasibility study, software requirements specification and the architectural system design diagrams without claiming completed deployment or measured project performance.

Contents

1 INTRODUCTION **1**

2 PROBLEM STATEMENT **3**

2.1 Problem Statement 3

2.2 Objectives 4

3 LITERATURE SURVEY **5**

3.1 Advancing Fake News Detection: Hybrid Deep Learning With FastText and Explainable AI 5

3.2 WELFake: Word Embedding Over Linguistic Features for Fake News Detection 6

3.3 FakeNewsNet: A Data Repository with Diverse Features in Social Media for Fake News Detection 6

3.4 Impact of Convolutional Neural Network and FastText Embedding on Text Classification 7

3.5 FakeBERT: Fake News Detection in Social Media using BERT and Deep Learning 7

3.6 A Systematic Survey on Explainable AI Applied to Fake News Detection . . 8

3.7 Exploiting Stance Similarity and Graph Neural Networks for Fake News Detection 8

3.8 Big Data ML-Based Fake News Detection Using Distributed Learning . . . 9

3.9 "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection 10

3.10 Fake News Detection: A Hybrid CNN-RNN Based Approach 10

3.11 Supervised Learning for Fake News Detection: An Explainable Approach . 11

4 PROPOSED SYSTEM **12**

4.1 Proposed Solution 12

4.2 Methodology and Workflow 13

5 FEASIBILITY STUDY 15
5.1 Technical Feasibility 15
5.2 Operational Feasibility 15
5.3 Economic Feasibility 16

6 SOFTWARE REQUIREMENTS SPECIFICATION 17
6.1 Overall Description 17
6.1.1 Project Perspective 17
6.1.2 Project Functions 17
6.2 User Classes and Characteristics 18
6.2.1 Operating Environment 18
6.2.2 Design and Implementation Constraints 18
6.2.3 Assumptions and Dependencies 19
6.3 External Interface Requirements 19
6.3.1 User Interfaces 19
6.3.2 Hardware Interfaces 19
6.3.3 Software Interfaces 19
6.4 Non-functional Requirements 20

7 SYSTEM DESIGN 21
7.1 Module 21
7.2 System Architecture 22
7.3 Sequence Diagram 23
7.4 Data Flow Diagram 25
7.4.1 Level 0 25
7.4.2 Level 1 26

8 SOFTWARE AND HARDWARE REQUIREMENT 27
8.1 Software Requirements 27
8.2 Hardware Requirements 28
8.2.1 Development and Model Training Requirements 28
8.2.2 Server and Deployment Requirements 29
8.2.3 Client Device Requirements 29

9 IMPLEMENTATION 30
9.1 System Overview and Interim Milestone Status 30
9.2 Core Implemented Components 31
9.3 Operational Verification Workflow 32
9.4 Prototype Walkthrough and Interface Results 33
9.4.1 Blank Input Dashboard State 33
9.4.2 Authentic News Veracity Classification 33
9.4.3 Deceptive / Fake News Veracity Classification 34
9.4.4 LIME Attribution Tooltip and Top 10 Feature Importance Chart . . 35
9.4.5 Model Specifications and Benchmark Modal 36
9.4.6 Mobile Device Responsiveness 36
9.5 Technical Setup of the Implemented Prototype 38

REFERENCES 39

List of Figures

7.1 System Architecture Diagram 23
7.2 UML Sequence Diagram 24
7.3 Data Flow Diagram: Level 0 (Context Diagram) 25
7.4 Data Flow Diagram: Level 1 26

9.1 Blank Input Dashboard Interface 33
9.2 Authentic News Veracity Classification and Attribution Heatmap 34
9.3 Fake News Veracity Classification and Keyword Highlighting 34
9.4 Top 10 Feature Importance Ranking and Attribution Tooltip 35
9.5 Model Architecture and Benchmark Metrics Modal 36
9.6 Mobile Responsive Interface Layout 37

Chapter 1

INTRODUCTION

Digital platforms and social media networks have become the primary sources of news for much of the world, enabling the rapid and largely unchecked spread of misinformation. Traditional fact-checking relies heavily on manual human verification, which is slow, labour-intensive and incapable of keeping pace with viral content. The consequences of unchecked fake news are serious: it distorts democratic elections, accelerates the spread of dangerous medical myths and has repeatedly incited communal violence. Both the United Nations and the World Health Organization have classified misinformation and information epidemics as critical threats to public health and democratic governance. Developing nations face acute vulnerability because large-scale internet adoption has outpaced digital media literacy.

Most contemporary automated classifiers achieve high predictive accuracy but operate as opaque black boxes that offer no insight into why a particular article was labelled real or fake. This opacity prevents journalists, fact-checkers and the general public from evaluating or trusting a verdict. The selected base paper, *Advancing Fake News Detection: Hybrid Deep Learning With FastText and Explainable AI* (Hashmi et al., IEEE Access, 2024), addresses both problems simultaneously. It combines FastText subword embeddings with a hybrid 1D-CNN and LSTM network, achieving up to 99 % accuracy on benchmark datasets and applies LIME to provide word-level attribution that makes the model reasoning visible.

The reviewed literature explores complementary methodologies. Verma et al. demonstrate the value of combining linguistic statistics with word embeddings in the WELFake corpus. Shu et al. provide a multi-dimensional social-context repository in FakeNewsNet. Umer et al. validate FastText subword representations for noisy text classification. Transformer architectures such as FakeBERT deliver strong semantic modelling but carry high computational and latency costs. Kumar and Chacko survey Explainable AI frameworks and underline the necessity of visual decision justifications. Additional works examine stance graphs, distributed learning pipelines, truthfulness benchmarks, hybrid sequential networks and supervised linguistic explainability.

The proposed project, **Ledger: AI-Based Fake News Detection System Using Explainable AI**, combines lightweight deep learning with interactive decision explainability. Rather than relying on external social-media graph data or expensive GPU infrastructure, Ledger operates as a standalone, text-based verification tool. Users submit a headline or full article text through a web interface and receive an instant authenticity verdict accompanied by highlighted word-level explanations.

The planned system consists of a FastText embedding layer, a 1D-CNN for local feature extraction, an LSTM for sequence modelling and a LIME attribution layer. A FastAPI server handles model serving and a React.js dashboard presents the result with confidence scores and highlighted keywords. The overall goal is to provide everyday citizens, journalists and fact-checkers with an accessible, interpretable and trustworthy fake-news verification assistant.

Chapter 2

PROBLEM STATEMENT

2.1 Problem Statement

The unchecked spread of fabricated news on digital platforms harms democratic processes, public health initiatives and social harmony. Existing automated detection approaches share several critical limitations.

First, nearly all current deep-learning classifiers function as black boxes. Even when they achieve high statistical accuracy they do not explain which words or phrases drove the prediction. Without interpretable justifications, users, journalists and regulatory bodies cannot assess whether a verdict is based on genuine linguistic indicators of deception or on spurious correlations in the training data.

Second, state-of-the-art contextual language models such as BERT require large GPU memories and produce high inference latency. These requirements make them impractical for low-cost, real-time consumer web applications where response times must remain sub-second on standard server hardware.

Third, social-media text is characterized by informal language, spelling errors, evolving internet slang and deliberate adversarial obfuscation. Standard word-embedding techniques assign fixed representations to complete tokens and fail when presented with out-of-vocabulary terms.

Fourth, many academic detection systems depend on external social-context attributes such as sharing graphs, retweet trees and comment sentiment. These dependencies make the models ineffective for newly published or standalone articles that lack prior social engagement.

The project therefore proposes Ledger, an AI-assisted web platform that addresses these limitations by combining FastText subword embeddings, a hybrid CNN-LSTM deep-learning model and LIME explainability to deliver real-time, text-only news verification with clear, word-level justifications.

2.2 Objectives

1. To design and develop an accessible web-based fake news detection system for real-time verification of news articles and headlines.
2. To use FastText subword n-gram embeddings to handle out-of-vocabulary tokens, typographical errors and informal digital slang.
3. To extract localized semantic features and deceptive phrase patterns using a 1D Convolutional Neural Network.
4. To model long-range sequential contextual dependencies using a Long Short-Term Memory network.
5. To integrate Explainable AI using LIME to produce transparent, word-level visual justifications for every classification verdict.
6. To evaluate and benchmark the hybrid framework on recognized open-access datasets including WELFake, FakeNewsNet and LIAR.
7. To construct a lightweight, high-throughput REST API using Python FastAPI for model serving and sub-second inference.
8. To develop an intuitive React.js user dashboard that displays the verdict, confidence score and an interactive highlighted explanation heatmap.

Chapter 3

LITERATURE SURVEY

The literature survey examines the selected base paper and ten complementary research works, focusing on hybrid deep learning, subword representations, contextual language models, social graph analytics, distributed learning and explainable AI for fake news detection. The limitations in each study identify technical gaps that motivated the design choices adopted in the Ledger platform.

3.1 Advancing Fake News Detection: Hybrid Deep Learning With FastText and Explainable AI

Authors: Ehtesham Hashmi, Sule Yildirim Yayilgan, Muhammad Mudassar Yamin, Subhan Ali and Mohamed Abomhara

Published On: IEEE Access, Volume 12, 2024

- **About:** Investigates the integration of hybrid deep learning with subword embeddings and LIME explainability to address the black-box limitation in fake news detection.
- **Methodology:** Text is preprocessed through regex cleaning, lemmatization and tokenization. FastText subword embeddings represent the tokens. A 1D-CNN extracts local n-gram features, an LSTM captures sequential dependencies and a Softmax layer classifies the text. LIME provides post-hoc word-level attribution.
- **Advantages:** Achieves 99 % accuracy on WELFake, 97 % on FakeNewsNet and 99 % on FakeNewsPrediction. FastText handles out-of-vocabulary terms and noisy social media text. LIME provides transparent word-level explanations that build user trust.
- **Limitations:** Evaluated primarily on English corpora; lacks multilingual support. Sensitive to superficial proper nouns and date references in short statements. Does not handle multimodal content such as manipulated images or synthetic video.

3.2 WELFake: Word Embedding Over Linguistic Features for Fake News Detection

Authors: Pawan Kumar Verma, Prateek Agrawal, Ivone Amorim and Radu Prodan

Published On: IEEE Transactions on Computational Social Systems, 2021

- **About:** Introduces WELFake, a large benchmark dataset and evaluates a hybrid framework combining linguistic statistics with dense word embeddings for fake news classification.
- **Methodology:** Aggregates 72,134 news records from four repositories. Concatenates 20 linguistic attributes with GloVe and Word2Vec embeddings and feeds the combined representation into ensemble and neural classifiers.
- **Advantages:** Provides one of the largest balanced open-access benchmark datasets, reducing overfitting. Achieves 96.73 % accuracy, outperforming standalone BERT and CNN by up to 4.25 %. Demonstrates strong cross-domain generalisation.
- **Limitations:** Manual extraction of 20 linguistic features is computationally expensive. Individual predictions lack interpretability explanations. Fixed embeddings cannot represent newly coined internet terminology.

3.3 FakeNewsNet: A Data Repository with Diverse Features in Social Media for Fake News Detection

Authors: Kai Shu, Deepak Mahudeswaran, Suhang Wang, Dongwon Lee and Huan Liu

Published On: Big Data, Mary Ann Liebert, 2020

- **About:** Presents a comprehensive data repository capturing news content, social network interactions and spatiotemporal propagation dynamics for fake news research.
- **Methodology:** Compiles verified news instances from PolitiFact and GossipCop, annotated with publisher credibility metadata, user engagement graphs and temporal dissemination cascades.
- **Advantages:** Contains 23,196 fact-checked articles with authoritative ground-truth labels. Supports joint exploration of news text and social propagation. Establishes an industry-standard benchmark for social context-aware research.

- **Limitations:** Suffers from incomplete historical social media data due to API rate limits and deleted posts. Heavy reliance on external social networks makes the dataset ineffective for evaluating newly published standalone articles.

3.4 Impact of Convolutional Neural Network and FastText Embedding on Text Classification

Authors: Muhammad Umer, Zainab Imtiaz, Muhammad Ahmad et al.

Published On: Multimedia Tools and Applications, 2023

- **About:** Investigates the combination of FastText subword embeddings and 1D Convolutional Neural Networks for classifying noisy and unstructured text.
- **Methodology:** Raw text is processed through FastText to capture character-level n-gram patterns. Parallel convolutional filters of varying kernel sizes extract local syntactic features, followed by global max-pooling and dropout regularisation.
- **Advantages:** Handles unknown vocabulary, typos and misspellings effectively. Significantly faster training and inference than Transformer models. Exceeds 94.5 % accuracy on benchmark classification tasks.
- **Limitations:** Focuses on localized phrase patterns and misses long-range semantic dependencies across long documents. Performance is sensitive to convolutional hyperparameter tuning.

3.5 FakeBERT: Fake News Detection in Social Media using BERT and Deep Learning

Authors: Rohit Kumar Kaliyar, Anurag Goswami and Pratik Narang

Published On: Soft Computing, Springer, 2021

- **About:** Proposes a deep-learning architecture combining bidirectional contextual representations from BERT with parallel convolutional layers for fake news detection.
- **Methodology:** Pre-trained BERT is fine-tuned to capture bidirectional semantics. The resulting embeddings pass through parallel 1D-CNN channels with different filter sizes before Softmax classification.

- **Advantages:** Models bidirectional sentence context alongside local semantic phrases. Achieves 98.90 % accuracy on benchmark corpora and is robust against subtle deceptive writing styles.
- **Limitations:** Requires heavy GPU compute and large memory bandwidth. Operates as an opaque black box with no decision transparency. High inference latency makes it unsuitable for sub-second web deployments.

3.6 A Systematic Survey on Explainable AI Applied to Fake News Detection

Authors: S. D. Madhu Kumar and Angel Mary Chacko

Published On: Engineering Applications of Artificial Intelligence, 2023

- **About:** Provides a structured analysis of Explainable AI frameworks applied to fake news detection, examining local, global and model-agnostic explanation approaches.
- **Methodology:** Categorises and evaluates post-hoc explanation techniques including LIME, SHAP, attention heatmaps and layer-wise relevance propagation across deep-learning architectures, assessing fidelity, stability and user trust.
- **Advantages:** Identifies key criteria for evaluating explanation quality. Highlights systemic biases in black-box fact-checking and emphasises the role of interpretable word attribution in building end-user credibility.
- **Limitations:** Perturbation-based explainers such as LIME introduce noticeable computational latency during real-time serving. No standardized quantitative metrics exist for benchmarking human trust in AI explanations.

3.7 Exploiting Stance Similarity and Graph Neural Networks for Fake News Detection

Authors: Kazuki Soga, Suguru Yoshida and Masahiro Muneyasu

Published On: Pattern Recognition Letters, Elsevier, 2024

- **About:** Investigates Graph Neural Networks that exploit user stance similarity and social propagation topology for news authenticity classification.

- **Methodology:** Constructs heterogeneous bipartite graphs capturing interactions between news articles and social media users. Edge weights reflect stance similarity from comments and Graph Convolutional Networks learn representation vectors over the graph structure.
- **Advantages:** Integrates collective crowd skepticism with news content. Improves F1-scores over standalone text models and shows enhanced resistance to adversarial text perturbations.
- **Limitations:** Requires extensive user-interaction and social-engagement records. Needs substantial memory and compute for graph propagation. Cannot evaluate cold-start articles that have zero social commentary.

3.8 Big Data ML-Based Fake News Detection Using Distributed Learning

Authors: Abdullah Altheneyan and Abdullah Alhadlaq

Published On: IEEE Access, 2023

- **About:** Evaluates distributed machine-learning pipelines built on Apache Spark for scalable fake news classification over large news streams.
- **Methodology:** Uses Spark MLlib for parallelised TF-IDF feature extraction and N-gram generation. Trains and ensembles Random Forest, Naive Bayes and Gradient Boosted Trees classifiers across distributed clusters.
- **Advantages:** Processes large volumes of enterprise news data efficiently. Achieves 92.4 % accuracy on the FNC-1 benchmark and provides high throughput suitable for big-data pipelines.
- **Limitations:** TF-IDF and N-gram features miss deeper semantic relationships and sequential context. Setting up and maintaining distributed Spark clusters incurs high operational overhead. Predictions lack explanatory attribution.

3.9 "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection

Authors: William Yang Wang

Published On: ACL, Association for Computational Linguistics, 2017

- **About:** Introduces the LIAR benchmark dataset providing fine-grained, multi-tier truthfulness ratings for political claim verification.
- **Methodology:** Compiles 12,800 short political statements from PolitiFact classified into six truthfulness levels. Evaluates standard machine-learning models, CNNs and bi-directional LSTMs using statement text and speaker metadata.
- **Advantages:** Provides an authoritative, human-annotated benchmark supporting nuanced multi-class veracity classification. Evaluates the impact of speaker history, party affiliation and context alongside lexical features.
- **Limitations:** Short statements frequently lack sufficient textual context for deep semantic understanding. Text-only models struggle without speaker metadata. Automated visual explanations are not provided.

3.10 Fake News Detection: A Hybrid CNN-RNN Based Approach

Authors: Jamal Abdul Nasir, Osama Subhani Khan and Iraklis Varlamis

Published On: International Journal of Information Management Data Insights, 2021

- **About:** Proposes a hybrid neural architecture combining Convolutional Neural Networks with Recurrent Neural Networks for automated news text classification.
- **Methodology:** Word embeddings pass through 1D-CNN layers to identify salient n-gram representations, followed by LSTM layers that preserve long-range contextual relationships. Dense layers generate binary veracity predictions.
- **Advantages:** Successfully models both localized phrase patterns and long-term sentence flow. Outperforms isolated CNN and RNN baselines on real-world corpora such as ISOT without requiring external social-media metadata.
- **Limitations:** Functions as a black-box model lacking built-in explanatory capabilities. Standard word embeddings struggle with vocabulary gaps and typos. Training latency

increases on very long articles.

3.11 Supervised Learning for Fake News Detection: An Explainable Approach

Authors: Julio C. S. Reis, André Correia, Fabrício Murai, Adriano Veloso and Fabricio Benevenuto

Published On: IEEE Internet Computing, 2019

- **About:** Explores explainable supervised learning for fake news detection by quantifying the predictive importance of linguistic, emotional and syntactic features.
- **Methodology:** Extracts handcrafted feature groups covering lexical diversity, readability scores, sentiment polarity and grammatical structures. Machine-learning classifiers are trained and global feature importance rankings are derived.
- **Advantages:** Identifies writing characteristics that strongly indicate deceptive text, confirming that sensational and emotionally charged vocabulary is a key marker of fake news. Provides globally interpretable feature rankings.
- **Limitations:** Relies heavily on manual feature engineering rather than automatic representation learning. Achieves lower accuracy on complex semantic sentences compared to modern neural networks. Cannot highlight suspicious words in individual articles in real time.

Chapter 4

PROPOSED SYSTEM

The proposed system, **Ledger**, is an Explainable AI-assisted web application designed to automate fake news verification with high classification accuracy, low inference latency and transparent decision interpretability. Traditional automated detection models predominantly function as black boxes that yield classifications without explaining why an article was flagged, limiting user credibility and public trust. Furthermore, existing Transformer-based language models demand substantial GPU resources that are impractical for lightweight, sub-second web deployments. Ledger addresses these challenges by combining subword semantic embeddings, a hybrid deep-learning classifier and local explanation heatmaps into a unified platform.

4.1 Proposed Solution

Ledger operates as a standalone, text-based verification tool that evaluates news articles and headlines based entirely on textual and linguistic characteristics. This eliminates dependencies on external social media networks, retweet graphs or user engagement histories, enabling the system to verify newly published standalone content immediately.

The core architecture combines FastText subword n-gram embeddings with a hybrid neural network composed of a 1D Convolutional Neural Network (CNN) and a Long Short-Term Memory (LSTM) network. FastText subword embeddings represent tokens as 300-dimensional dense vectors derived from character n-grams. This enables the model to capture morphological subtleties and effectively resolve out-of-vocabulary terms, typographical errors, informal internet slang and deliberate adversarial spelling variations.

The 1D-CNN layer employs parallel convolutional filters with varying kernel sizes to detect salient local n-gram features and deceptive syntactic structures within the text. The extracted feature representations are sequentially processed by the LSTM network, which captures long-range contextual dependencies and preserves semantic coherence across complete paragraphs. A dense Softmax layer computes class probabilities to output a binary

veracity verdict (Real or Fake) alongside a calibrated confidence score.

To ensure decision transparency, the system integrates Local Interpretable Model-agnostic Explanations (LIME). When a prediction is generated, LIME creates localized perturbations of the input text and fits an interpretable linear surrogate model. This computes token-level attribution weights that visually highlight influential words within the user interface, distinguishing between words supporting authenticity and those indicating deception.

The primary capabilities of the proposed Ledger system are summarized below:

1. **Real-Time Text Verification:** A streamlined web platform allowing users to paste news text or headlines and obtain sub-second authenticity verdicts.
2. **Subword Semantic Representation:** FastText character-level embeddings resolve spelling mistakes, emerging digital slang and unknown vocabulary without information loss.
3. **Dual-Stage Hybrid Neural Learning:** 1D-CNN captures localized deceptive phrase patterns while LSTM preserves long-range semantic context across the article.
4. **Explainable AI and Transparency:** LIME generates word-level attribution scores, providing visual heatmaps that explain why a specific verdict was reached.
5. **Independent Text-Only Verification:** Classifies articles without requiring external social-media propagation trees, comment histories or user engagement metadata.
6. **Lightweight and Cost-Effective Deployment:** Operates on commodity CPU hardware without requiring expensive GPU servers, ensuring economical scalability.
7. **Interactive User Dashboard:** Displays the classification label, numerical confidence percentage and an interactive word-attribution heatmap.

4.2 Methodology and Workflow

The end-to-end verification methodology comprises seven sequential stages:

1. **Input Stage:** The user accesses the web dashboard and submits unverified news text or a headline through the input form.
2. **Text Preprocessing and Cleaning:** The backend cleans the input string by stripping HTML tags, removing non-alphanumeric noise, normalizing whitespace and applying

lemmatization and tokenization.

3. **Subword Embedding Generation:** Preprocessed tokens are mapped into 300-dimensional dense vectors using pre-trained FastText subword models.
4. **Feature Extraction (CNN):** Parallel 1D-CNN filters extract salient n-gram phrases and deceptive syntactic indicators.
5. **Sequential Context Modeling (LSTM):** Recurrent LSTM memory cells capture long-range sequential dependencies across the feature sequence.
6. **Veracity Classification:** A dense layer with Softmax activation produces the binary prediction label (Real or Fake) with a calibrated confidence percentage.
7. **Explanation and Visualization (LIME):** The LIME explainer computes token-level attribution weights and the web interface renders color-coded word highlights alongside the final prediction verdict.

Chapter 5

FEASIBILITY STUDY

The objective of this feasibility study is to evaluate whether the proposed fake news detection system can be successfully developed, implemented and deployed within practical resource constraints. The assessment examines technical, operational and economic dimensions to ensure system practicality and viability.

5.1 Technical Feasibility

The proposed system is technically feasible as it leverages well-established, industry-standard technologies and open-source frameworks. The machine learning pipeline is built with Python, utilizing FastText for subword embeddings, PyTorch or TensorFlow for the hybrid CNN-LSTM neural network and the official LIME library for post-hoc interpretability.

Unlike heavy Transformer architectures such as BERT or RoBERTa that require multi-gigabyte GPU memory and complex tensor parallelization, the FastText + 1D-CNN + LSTM pipeline is computationally compact. Model inference and localized LIME perturbation require minimal processing overhead and can execute comfortably on standard multicore x86-64 CPUs with sub-second response times.

The application architecture decouples the frontend client from the backend inference service using a RESTful API built on Python FastAPI and Uvicorn. This asynchronous architecture ensures rapid request routing and predictable resource consumption. The technical viability of the hybrid deep-learning approach is strongly substantiated by the base paper (Hashmi et al., 2024), which demonstrated up to 99 % accuracy on the WELFake dataset and 97 % on FakeNewsNet.

5.2 Operational Feasibility

Ledger is operationally feasible as it addresses the practical operational requirements of everyday news readers, journalists and professional fact-checkers. Traditional detection

tools often encounter resistance because opaque black-box verdicts fail to provide actionable justifications. By delivering word-level attribution heatmaps, Ledger enables users to immediately evaluate the linguistic evidence supporting a prediction, establishing institutional trust and credibility.

From an end-user standpoint, the system requires no specialized technical expertise. Users simply paste text into a web browser and receive an immediate verdict accompanied by intuitive color-coded word highlights. The application operates as a self-contained service that does not impose complex installation steps or configuration burdens on users. Furthermore, because Ledger performs text-only verification without relying on external social media graphs, it remains fully operational for newly written or unshared articles.

5.3 Economic Feasibility

The development, deployment and maintenance of the Ledger platform are highly cost-effective. The entire software stack is constructed from open-source technologies, including Python, FastAPI, React.js, FastText, PyTorch and LIME, incurring zero proprietary licensing expenditures.

Public benchmark datasets utilized for model training and evaluation, including WELFake, FakeNewsNet and LIAR, are openly accessible for academic and development purposes. Furthermore, the lightweight computational footprint of the FastText + CNN-LSTM architecture eliminates the requirement for dedicated, high-cost cloud GPU instances during inference. The system can be hosted on standard commodity virtual servers, keeping recurring operational and hosting costs to an absolute minimum while delivering dependable verification services.

Chapter 6

SOFTWARE REQUIREMENTS SPECIFICATION

The Software Requirements Specification (SRS) establishes the complete functional and non-functional requirements governing the design, implementation and deployment of the Ledger fake news detection system.

6.1 Overall Description

6.1.1 Project Perspective

Ledger is a self-contained, web-based fake news verification system built on a client-server architecture. The system integrates an interactive frontend interface with a high-performance backend machine-learning server. The React.js frontend provides the user interface for text input and visualizes verification results alongside explanation heatmaps. The Python FastAPI backend orchestrates text cleaning, vector embedding lookup, deep-learning inference and LIME feature attribution. The entire architecture operates independently of external social-network platforms, functioning as a standalone text evaluation service.

6.1.2 Project Functions

The core functions provided by the Ledger system include:

- Accepting arbitrary news headlines and article passages through a responsive web interface.
- Preprocessing input text through regex cleaning, lowercasing, lemmatization and tokenization.
- Converting tokens into 300-dimensional FastText subword embeddings to handle typos and slang.
- Classifying text as Real or Fake using a hybrid 1D-CNN and LSTM deep-learning model.
- Calculating calibrated confidence percentages for the classification verdict.
- Performing word-level attribution using LIME to quantify the contribution of individual tokens.

- Displaying an interactive color-coded explanation heatmap in the client web dashboard.
- Providing a high-throughput REST API endpoint for automated and programmatic verification.

6.2 User Classes and Characteristics

The system identifies three primary user classes:

- **General Public and News Consumers:** Require an accessible, intuitive interface to verify suspicious headlines or social-media posts without needing technical background.
- **Journalists and Fact-Checkers:** Require rapid veracity screening and transparent word-level evidence to assist professional verification workflows.
- **System Administrators and Researchers:** Manage model deployment, evaluate classification performance metrics and maintain server operations.

6.2.1 Operating Environment

- **Client Environment:** Any modern web browser including Google Chrome, Mozilla Firefox, Apple Safari and Microsoft Edge running on desktop, laptop or mobile devices.
- **Server Operating System:** Linux (Ubuntu 22.04 LTS or higher) or Windows 10/11 64-bit operating system.
- **Server Runtime Environment:** Python 3.10 or higher with Uvicorn ASGI server and Node.js version 18 or higher for frontend asset management.
- **Hardware Environment:** Commodity x86-64 host with a minimum of 4 CPU cores, 8 GB RAM and 20 GB available storage space.

6.2.2 Design and Implementation Constraints

- **Programming Languages and Frameworks:** The backend is developed in Python using FastAPI, while the frontend dashboard is constructed with React.js.
- **Model Complexity:** The hybrid CNN-LSTM neural network must remain computationally compact to ensure inference and LIME perturbations execute efficiently on standard CPUs.
- **Stateless Architecture:** The verification pipeline processes requests in-memory without requiring mandatory persistent relational databases for user tracking.

- **Network Dependency:** Client devices require standard HTTP/HTTPS connectivity to communicate with the backend API.

6.2.3 Assumptions and Dependencies

- Input articles are written primarily in the English language, matching the training domain of the benchmark datasets.
- Submitted articles provide sufficient textual content (headline or body excerpt) to allow meaningful linguistic feature extraction.
- Standard open-source libraries (FastText, LIME, FastAPI, PyTorch) remain supported and available in the execution environment.

6.3 External Interface Requirements

6.3.1 User Interfaces

The web interface is designed to be clean, responsive and intuitive. It includes a multiline text submission field, an instant verification button with dynamic loading feedback, a verdict summary card showing the binary prediction badge (Real or Fake) with an associated confidence percentage bar and an interactive heatmap viewer that renders words with green (supporting real) or red (indicating fake) background highlights.

6.3.2 Hardware Interfaces

The system communicates over standard TCP/IP networking hardware (Ethernet or Wi-Fi). No specialized external hardware interfaces, proprietary sensors or GPU accelerators are mandatory for system operation.

6.3.3 Software Interfaces

The system uses a RESTful JSON API over HTTP/HTTPS for communication between the React.js client and the FastAPI backend. Incoming payloads are validated against Pydantic schemas. Model weights and pre-trained FastText subword matrices are loaded from disk storage into memory during server startup.

6.4 Non-functional Requirements

- **Performance:** Verification requests, including deep-learning classification and LIME explanation generation, must complete within 2 seconds on standard server hardware.
- **Usability:** The user interface must present clear verdicts and self-explanatory visual highlights requiring no prior training or technical knowledge.
- **Reliability:** The backend must handle invalid, empty or excessively long inputs gracefully with descriptive HTTP error responses.
- **Scalability:** The asynchronous ASGI server architecture must support concurrent verification requests without memory leaks or request timeouts.
- **Maintainability:** A modular directory structure cleanly separates frontend components, REST API endpoints, preprocessing scripts and neural model modules.
- **Security:** Input sanitization prevents cross-site scripting (XSS) and code injection, while communication occurs over secure HTTPS channels.

Chapter 7

SYSTEM DESIGN

System design serves as the blueprint for translating functional requirements and theoretical concepts into an operational software architecture. This chapter presents the functional modules and design diagrams for the Ledger fake news detection system. The visual representations delineate the multi-tier system topology, the hierarchical movement of data through progressive processing abstractions and the chronological interaction lifecycle among distributed components.

7.1 Module

The Ledger fake news detection system is organized into six functional modules that collaborate to deliver end-to-end, interpretable verification:

1. **User Interface Module (React.js Frontend):** Provides an intuitive, responsive web dashboard where users input news headlines or articles. It performs client-side validation, dispatches asynchronous verification requests to the backend API and renders the output verdict badge, confidence percentage and an interactive word-attribution heatmap.
2. **API Service and Orchestration Module (FastAPI Backend):** Exposes high-performance RESTful API endpoints. Built with Python FastAPI and served via Uvicorn, this module validates incoming request payloads, orchestrates data transfer between preprocessing routines, model inference and explainability engines and formats results into structured JSON payloads.
3. **Text Preprocessing and Normalization Module:** Cleans and normalizes submitted text before representation learning. Operations include regex-based stripping of HTML tags, punctuation normalization, conversion to lowercase, tokenization and lemmatization to remove noise and standardize input tokens.
4. **Feature Extraction and Subword Embedding Module (FastText):** Transforms preprocessed tokens into 300-dimensional dense vectors using character-level n-gram subword

embeddings. This module handles out-of-vocabulary terms, typographical errors and informal social-media slang by constructing representations from constituent character n-grams.

5. **Hybrid Neural Classification Module (1D-CNN and LSTM):** Executes binary veracity classification over the embedding sequences. Multiple 1D-CNN convolutional kernels extract localized n-gram syntax and deceptive phrase patterns, followed by LSTM recurrent layers that capture long-range contextual dependencies across the text. A final dense layer with Softmax activation generates class probabilities for Real and Fake categories.
6. **Explainable AI Attribution Module (LIME):** Generates post-hoc interpretability for each classification verdict. By systematically perturbing input words and measuring prediction variances, LIME fits a localized linear surrogate model to determine positive and negative attribution weights for individual tokens, enabling visual keyword highlighting.

7.2 System Architecture

The system architecture follows a modular three-tier structure comprising the presentation tier, the backend application tier and the core artificial intelligence engine tier. The architectural design guarantees loose coupling, distinct separation of concerns and efficient horizontal scalability.

The presentation tier is implemented as a responsive web dashboard using React.js. It provides an intuitive interface for journalists, fact-checkers and everyday users to submit arbitrary news articles or headlines for verification. The interface receives user input, performs client-side validation and renders the computed authenticity verdict alongside confidence ratings and an interactive word-importance heatmap.

The backend application tier is constructed with Python FastAPI and served via an asynchronous ASGI server. It exposes high-performance RESTful API endpoints for news verification requests. The backend validates incoming payloads, orchestrates data transfer between the frontend and the machine learning pipeline and formats output predictions into standardized JSON responses.

The core AI engine tier hosts the natural language processing preprocessor, the FastText embedding module, the hybrid 1D-CNN and LSTM neural network and the

LIME explainability module. FastText subword embeddings represent input tokens as 300-dimensional dense vectors. The 1D-CNN layers extract localized n-gram syntax and deceptive phrase patterns, while the LSTM layers capture long-range contextual dependencies across the article. Finally, the LIME explainer perturbs the input text and computes token-level attribution weights to explain the classification verdict.

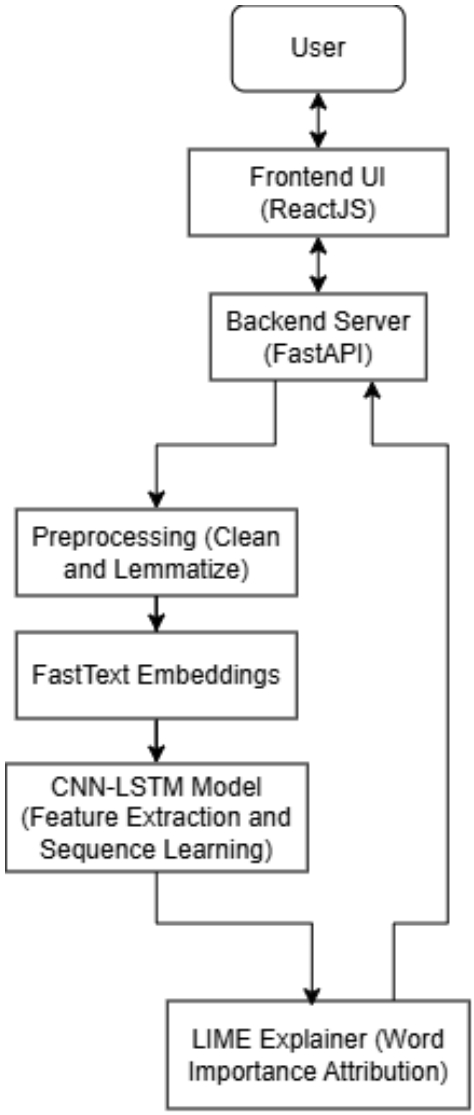


Figure 7.1: System Architecture Diagram

7.3 Sequence Diagram

The UML Sequence Diagram illustrates the chronological message exchange between system participants during a complete news verification lifecycle. The diagram models

six collaborating lifelines: the User, the React.js Frontend, the FastAPI Backend, the Text Preprocessor and FastText Vectorizer, the CNN-LSTM Classifier and the LIME Explainer.

The sequence begins when the user submits news text and triggers the verification action on the web interface. The React frontend sends an asynchronous HTTP POST request to the FastAPI verification endpoint. The backend invokes the preprocessing and FastText vectorizer module, which tokenizes the text and generates dense subword tensors before returning them to the API handler.

The backend forwards the tensor representations to the CNN-LSTM classification model, which computes the class likelihoods and returns the binary verdict alongside the confidence percentage. To ensure transparency, the backend dispatches the input text and model handle to the LIME explainer. LIME performs iterative word masking, queries the model for surrogate inference on perturbed variants and calculates localized word importance weights.

Once attribution weights are computed, LIME returns the attribution dictionary to the backend. The FastAPI server packages the verdict, confidence score and word highlights into a structured JSON payload and transmits it to the frontend client. The React interface parses the payload and dynamically updates the dashboard, presenting the user with a clear authenticity verdict and an interactive highlighted textual explanation.

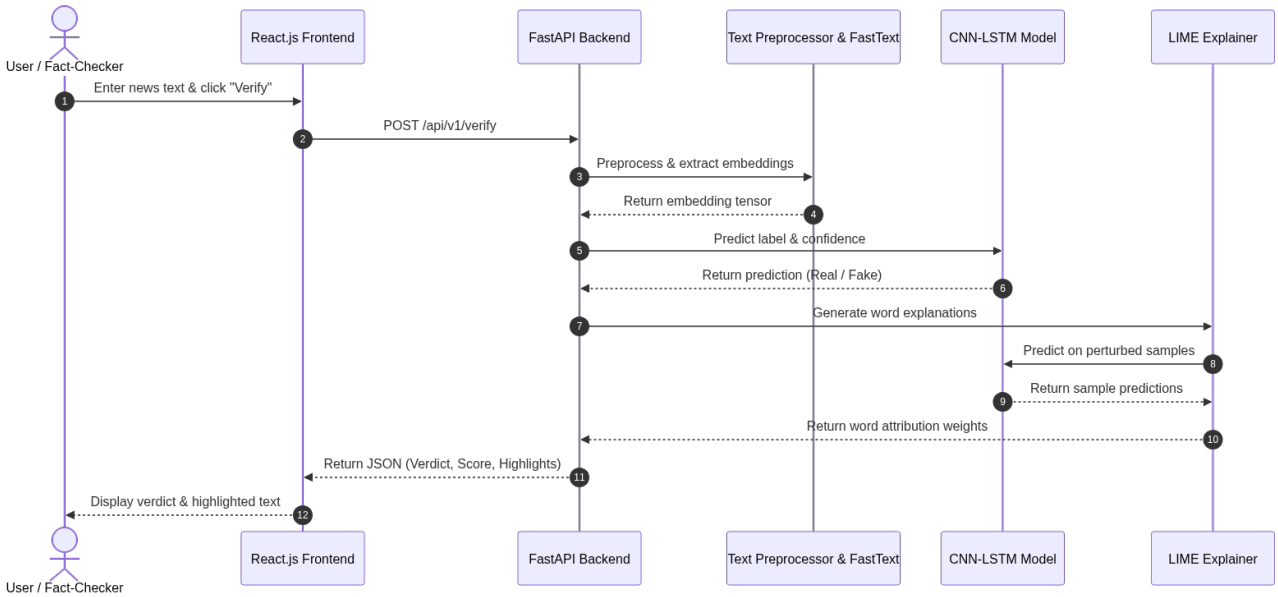


Figure 7.2: UML Sequence Diagram

7.4 Data Flow Diagram

A Data Flow Diagram (DFD) provides a graphical representation of the flow of data through an information system, tracing the path from external data inputs to processed outputs. DFDs serve as a primary analytical tool in structured system design by functionally decomposing complex operations into distinct hierarchical levels of abstraction. External entities represent sources or destinations of information, directed arrows illustrate data movement pathways, processes depict computational or algorithmic transformations applied to data and data stores denote persistent or in-memory repositories. In the Ledger architecture, data flow diagrams characterize how submitted news articles are ingested from client interfaces, channeled through natural language preprocessing and FastText embedding routines, classified by the hybrid neural network and interpreted via LIME explanations to deliver accountable authenticity verdicts to end users.

7.4.1 Level 0

The Data Flow Diagram Level 0, also designated as the context diagram, defines the system boundary by illustrating the primary interactions between external entities and the Ledger system as a single unified process.

Two principal external entities interact with the system: the User (or Fact-Checker) and the Machine Learning Engineer (or System Administrator). The user submits unverified news text or article headlines to the Ledger system. In return, the system provides an authenticity assessment comprising the predicted label (Real or Fake), an associated confidence percentage and word-level explainability highlights indicating the linguistic basis for the prediction.

The machine learning engineer supplies annotated benchmark datasets, such as WELFake, FakeNewsNet and LIAR, for offline model training and fine-tuning. The system outputs training convergence metrics, cross-validation performance summaries and loss statistics to allow the engineer to monitor model reliability.

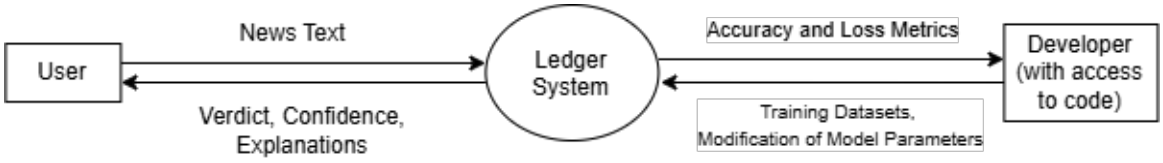


Figure 7.3: Data Flow Diagram: Level 0 (Context Diagram)

7.4.2 Level 1

The Data Flow Diagram Level 1 decomposes the unified Ledger system into four modular sub-processes, establishing the precise transformation of data from raw user input to an explainable prediction.

Process 1.0 (Preprocess Text) receives raw text from the user and applies cleaning, tokenization and lemmatization routines. Noise such as extraneous whitespace, HTML tags and non-alphanumeric characters is removed to generate normalized token streams.

Process 2.0 (Generate Embeddings) takes the normalized tokens and retrieves corresponding subword vector representations from the pre-trained FastText embedding repository. This step ensures that out-of-vocabulary terms, typographical errors and internet slang are projected into meaningful semantic vector spaces.

Process 3.0 (Classify News) feeds the dense embedding tensors into the trained CNN-LSTM neural network. Pre-trained weights stored in the model repository evaluate local syntactic patterns and sequential semantic flow, producing class probabilities for the Real and Fake categories.

Process 4.0 (Generate Explanation) receives the prediction from the classification process and invokes the LIME explainer. By perturbing words within the input sample and fitting a localized linear surrogate model, the process calculates positive and negative attribution weights for individual tokens. The final package, combining the veracity verdict, confidence score and word-level attribution highlights, is transmitted back to the user interface.

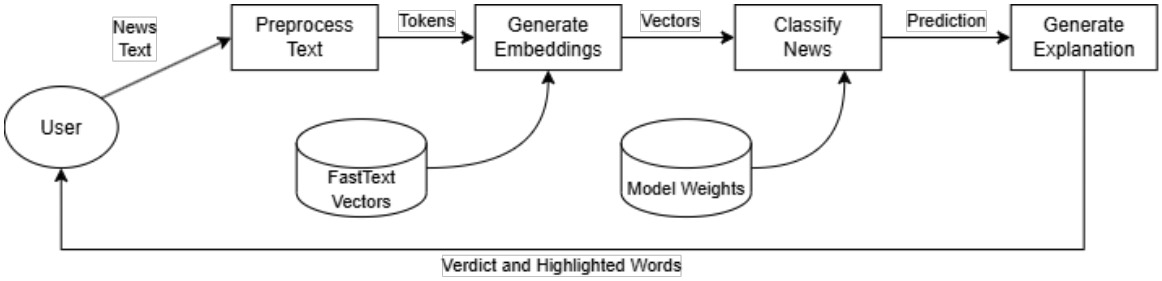


Figure 7.4: Data Flow Diagram: Level 1

Chapter 8

SOFTWARE AND HARDWARE REQUIREMENT

The implementation and execution of the Ledger fake news detection system depend on specific software frameworks, runtime environments and hardware platforms. This chapter outlines the complete software and hardware requirements necessary for development, model training, server deployment and client access.

8.1 Software Requirements

The software ecosystem comprises machine learning libraries, natural language processing toolkits, web application frameworks and developer utilities:

- **Operating System:** Linux (Ubuntu 22.04 LTS or higher) or Microsoft Windows 10/11 (64-bit operating system).
- **Programming Languages:** Python 3.10 or higher for backend service logic, text preprocessing and machine learning pipelines; JavaScript (ECMAScript 2022+) for interactive frontend components.
- **Machine Learning Frameworks:** PyTorch 2.0+ or TensorFlow/Keras 2.x for neural network modeling, tensor computations and model serialization.
- **Natural Language Processing Libraries:** FastText library for extracting 300-dimensional subword n-gram embeddings; Natural Language Toolkit (NLTK) and SpaCy for sentence tokenization, regex cleaning, stopword filtering and lemmatization.
- **Explainable AI (XAI) Framework:** LIME (Local Interpretable Model-agnostic Explanations) toolkit for generating local perturbation instances and calculating token-level feature importance attribution weights.
- **Backend Framework and Web Server:** Python FastAPI for high-throughput asynchronous RESTful API endpoints; Uvicorn ASGI production server for event-driven concurrency; Pydantic for schema validation and request-response serialization.
- **Frontend Framework and Tools:** React.js (version 18 or higher) for constructing the

single-page user interface; Node.js (version 18 LTS or higher) and npm/yarn for package management; Axios for handling asynchronous HTTP requests.

- **Development and Collaboration Tools:** Visual Studio Code as the primary integrated development environment (IDE); Google Colab (with cloud GPU acceleration) and Jupyter Notebook for iterative exploratory data analysis and model training; Git and GitHub for distributed version control and repository management.
- **Benchmark Datasets:** WELFake dataset (comprising 72,134 verified news articles) supplemented by FakeNewsNet and LIAR benchmark corpora for training and comparative evaluation.

8.2 Hardware Requirements

The hardware requirements are categorized into development and model training specifications, server hosting specifications and end-user client device requirements.

8.2.1 Development and Model Training Requirements

Model training, embedding extraction and hyperparameter tuning involve computationally demanding matrix operations:

- **Processor (CPU):** Multi-core processor (Intel Core i5/i7 12th Generation or AMD Ryzen 5/7 equivalent, minimum 6 physical cores operating at 2.5 GHz or higher).
- **System Memory (RAM):** Minimum 16 GB DDR4/DDR5 RAM to handle large tokenized text arrays and dataset batches without memory exhaustion.
- **Storage:** Minimum 50 GB available Solid State Drive (SSD) storage to accommodate raw and preprocessed datasets, the pre-trained FastText binary model (`cc.en.300.bin`) and saved neural network checkpoint files.
- **Graphics Processing Unit (GPU):** Dedicated NVIDIA GPU with minimum 8 GB VRAM (supporting CUDA and cuDNN) or cloud-hosted NVIDIA Tesla T4 GPU (16 GB GDDR6 VRAM provided via Google Colab) to accelerate tensor convolutions and recurrent backpropagation passes.

8.2.2 Server and Deployment Requirements

The production server hosts the FastAPI application, loads the trained CNN-LSTM weights and FastText matrices into memory and services incoming inference requests:

- **Processor (CPU):** x86-64 multi-core server processor (minimum 4 cores operating at 2.2 GHz or higher).
- **System Memory (RAM):** Minimum 8 GB RAM (16 GB recommended) to retain embedding vectors and model weights in resident memory for low-latency inference.
- **Storage:** Minimum 20 GB available SSD storage for application runtimes, logs and static assets.
- **Network Interface:** Standard 100/1000 Mbps Ethernet network adapter with stable high-speed broadband internet connectivity.

8.2.3 Client Device Requirements

The frontend web application is lightweight and offloads all compute-intensive NLP operations to the backend API:

- **Device Type:** Any desktop computer, laptop, tablet or smartphone.
- **Processor:** Standard dual-core processor (1.5 GHz or higher).
- **System Memory (RAM):** Minimum 2 GB RAM (4 GB or more recommended for smooth browser operation).
- **Display Resolution:** Minimum 1024×768 pixels (1920×1080 recommended for optimal visualization of attribution heatmaps).
- **Web Browser:** Any standards-compliant modern web browser including Google Chrome, Mozilla Firefox, Apple Safari or Microsoft Edge with JavaScript execution enabled.
- **Network Connectivity:** Active broadband, Wi-Fi or cellular (4G/5G) internet connection.

Chapter 9

IMPLEMENTATION

The implementation phase translates the theoretical architecture, system design models and software requirements into functional software artifacts. In accordance with the academic project milestones, the system is currently at the Phase I interim stage, representing approximately 25 % overall project completion. Under the prescribed interim evaluation guidelines, primary emphasis is placed on delivering a fully functional, high-fidelity user interface, an interactive Explainable AI visualization pipeline and an autonomous client-side verification engine. This chapter presents the implemented frontend prototype, the visual explainability mechanisms, the user interaction workflow and the graphical interface results captured across standard evaluation scenarios.

9.1 System Overview and Interim Milestone Status

The Ledger platform is designed as an accessible, text-based fake news detection system that pairs deep-learning classification with word-level Explainable AI. At the current 25 % completion milestone, implementation efforts have focused on the presentation layer and client-side verification pipeline. This prioritization ensures that news readers, journalists and evaluators can immediately interact with the verification workflow, evaluate the visual interpretability of predictions and inspect veracity heatmaps prior to backend deep-learning model training.

The client-side application is implemented as a responsive Single-Page Application (SPA) developed using the React.js framework, Vite build tooling and Tailwind CSS. To enable autonomous operation and verification during early evaluation, the frontend incorporates a dedicated Mock Verification Engine (`mockData.js`). This engine simulates the inference output of the planned FastText and hybrid CNN-LSTM neural pipeline by supplying calibrated veracity predictions and token-level attribution weights derived from verified benchmark news corpora. Consequently, the user interface operates as a standalone system capable of demonstrating realistic verification scenarios, interactive heatmap rendering and feature

importance ranking without requiring active backend server dependencies.

9.2 Core Implemented Components

The implemented prototype consists of six modular components integrated within the React.js single-page architecture:

1. **Verification Input and Preset Selection Module:** Provides an expansive text area for submitting arbitrary news headlines or full article passages. The component features dynamic character and token counters that monitor submission length in real time. To facilitate rapid demonstration and standardized evaluation, a preset selector allows evaluators to load curated benchmark passages representing verified authentic news, fabricated political claims and health misinformation with a single click. Action controls include an instant verification trigger with animated loading indicators and a text clearing button.
2. **Autonomous Mock Verification Engine:** Acts as the client-side data provider during the interim milestone. It encapsulates domain-specific veracity profiles based on verified literature cases, returning binary classification verdicts (`Authentic News` or `Deceptive / Fake News`), numerical confidence percentages, class probability distributions and signed LIME token attribution weights.
3. **Verdict and Confidence Gauge Card:** Renders the verification outcome through high-contrast visual cues. Authentic news is marked with an emerald green badge and deceptive content is flagged with a prominent crimson red badge. The card incorporates an animated horizontal confidence meter displaying the percentage certainty, a statistical probability breakdown for both classes and execution latency metadata.
4. **Explainable AI Heatmap Visualization Component:** Implements an interactive token-level explanation view that renders the submitted article with color-coded background highlights. Words that provide evidence of authenticity are highlighted in green tones, whereas terms associated with deceptive patterns are highlighted in red tones. Highlight intensity scales proportionally with the absolute numerical attribution score. Hovering the cursor over any highlighted word activates an interactive tooltip displaying the exact LIME score alongside a qualitative indicator explaining whether the term supports authenticity

or indicates deception.

5. **Feature Importance Ranking Component:** Constructs a horizontal bilateral bar chart visualizing the ten most influential words sorted by absolute attribution magnitude. A centered zero-axis delineates the two classes: red bars extend leftward for deceptive keywords and green bars extend rightward for credibility-supporting terms. Numerical attribution values are rendered adjacent to each bar to provide transparent quantitative justifications.
6. **Model Specifications and Benchmark Modal:** An educational overlay accessible from the primary navigation bar. It presents the target neural architecture specifications (FastText 300-dimensional embeddings, two 1D-CNN layers with kernels 4 and 3, two LSTM layers with 50 and 30 memory units, Dropout and Softmax classification) alongside benchmark classification metrics reported in the base literature across the WELFake, FakeNewsNet and LIAR datasets.

9.3 Operational Verification Workflow

The operational workflow implemented in the client dashboard follows a four-step sequence:

1. **Input Entry:** The user enters unverified news text into the submission area or selects one of the pre-configured benchmark article presets.
2. **Verification Dispatch:** Upon clicking the verification button, the interface validates that the input text contains adequate content, engages an animated loading state and dispatches the payload to the verification engine.
3. **Veracity Evaluation and Rendering:** The verification engine evaluates the text tokens, computes class probabilities and returns the verdict badge, confidence score and token attribution weights.
4. **Interactive Explainability Exploration:** The dashboard renders the color-coded LIME word heatmap and the bilateral feature importance ranking chart. The user can inspect individual tokens using mouse hover tooltips to understand the linguistic basis of the decision.

9.4 Prototype Walkthrough and Interface Results

9.4.1 Blank Input Dashboard State

Figure 9.1 illustrates the initial dashboard view prior to executing verification. The header displays active model metadata, the hero section outlines the core performance targets and the input container provides article preset loaders with real-time character counters.

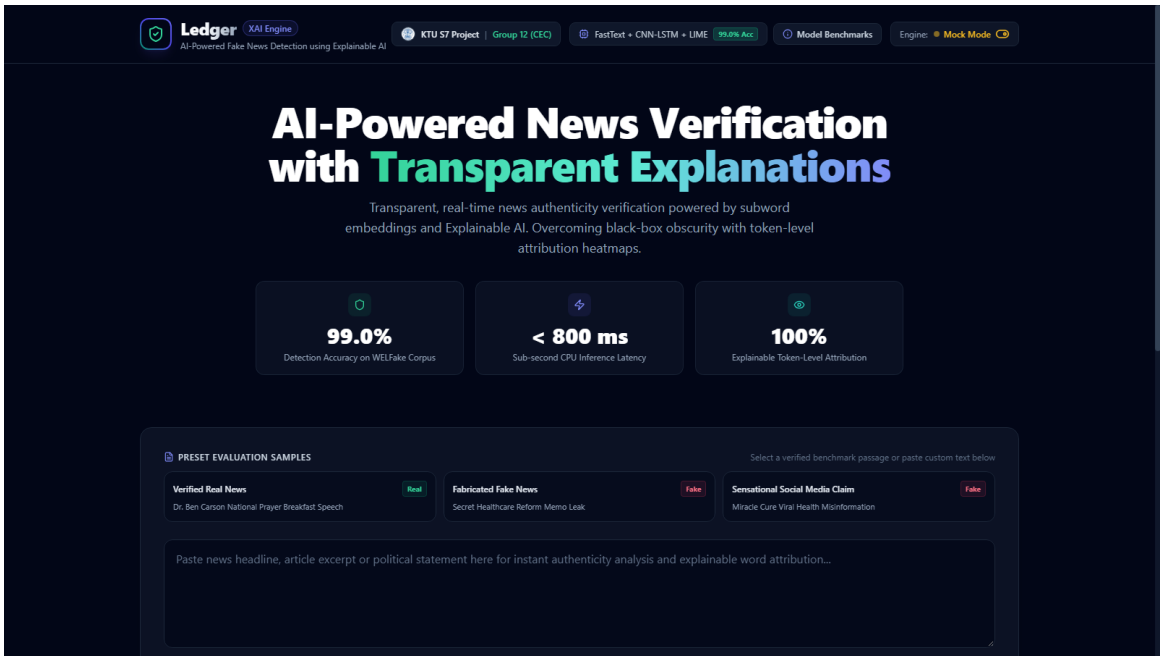


Figure 9.1: Blank Input Dashboard Interface

9.4.2 Authentic News Veracity Classification

Figure 9.2 demonstrates the verification of a legitimate political news report (Sample 1: Verified Real News). The interface outputs an emerald green `AUTHENTIC NEWS` badge with a 98.6 % confidence score and highlights credible anchor words in green tones below. The number of tokens and characters is also displayed below the text entry box. The confidence percentage for probabilities of both real and fake is also displayed.

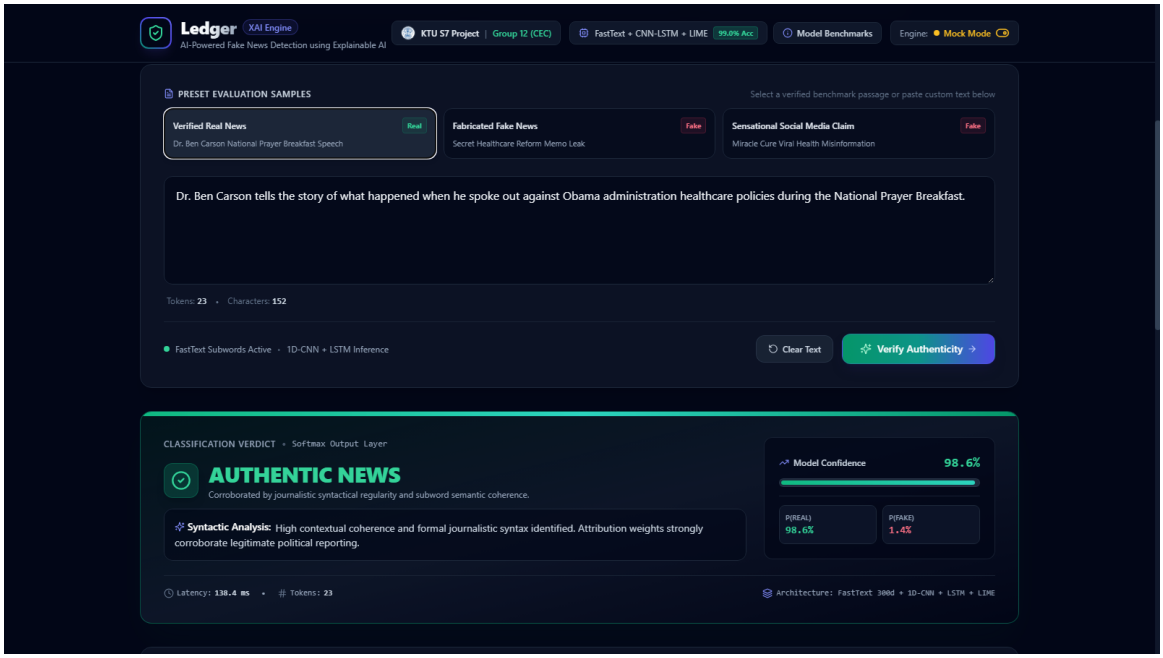


Figure 9.2: Authentic News Veracity Classification and Attribution Heatmap

9.4.3 Deceptive / Fake News Veracity Classification

Figure 9.3 depicts the evaluation of fabricated content (Sample 2: Fabricated Fake News). The dashboard displays a crimson red `DECEPTIVE / FAKE NEWS` badge with a 99.2 % confidence rating, highlighting sensationalized deception markers in red.

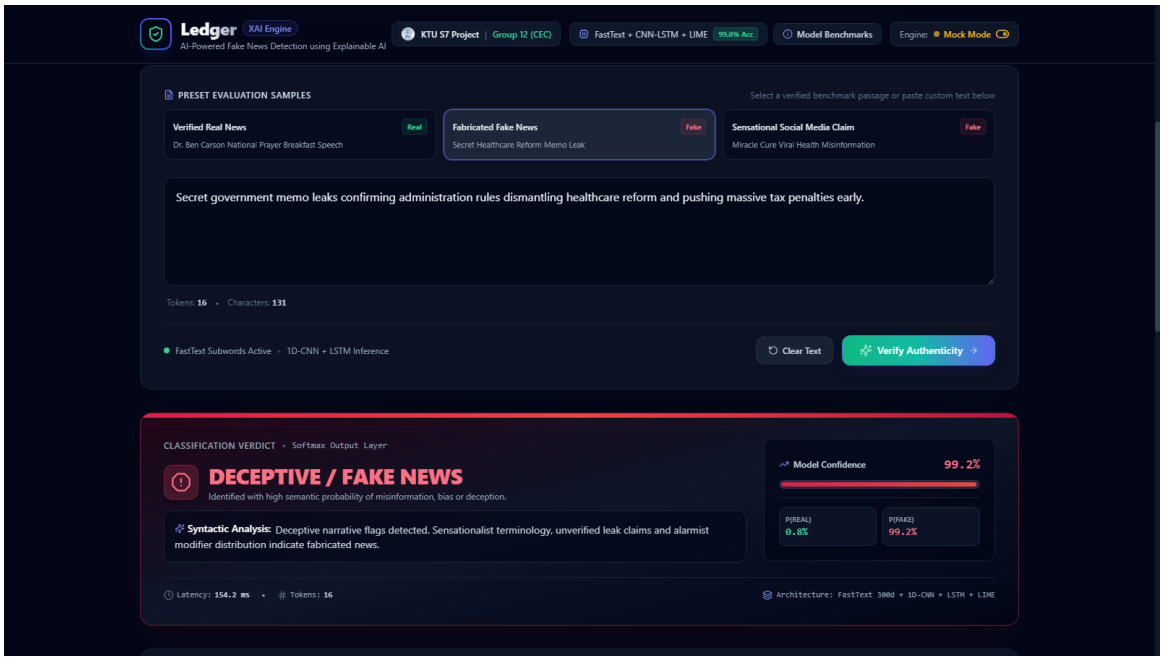


Figure 9.3: Fake News Veracity Classification and Keyword Highlighting

9.4.4 LIME Attribution Tooltip and Top 10 Feature Importance Chart

Figure 9.4 demonstrates the Explainable AI attribution mechanisms. Hovering over a flagged token reveals its exact LIME (Local Interpretable Model-agnostic Explanations) score in a tooltip, revealing its individual attribution score. The bilateral chart ranks the top ten influential words extending left for deceptive cues (negative or fake indication) and right for credibility (positive or real indication).

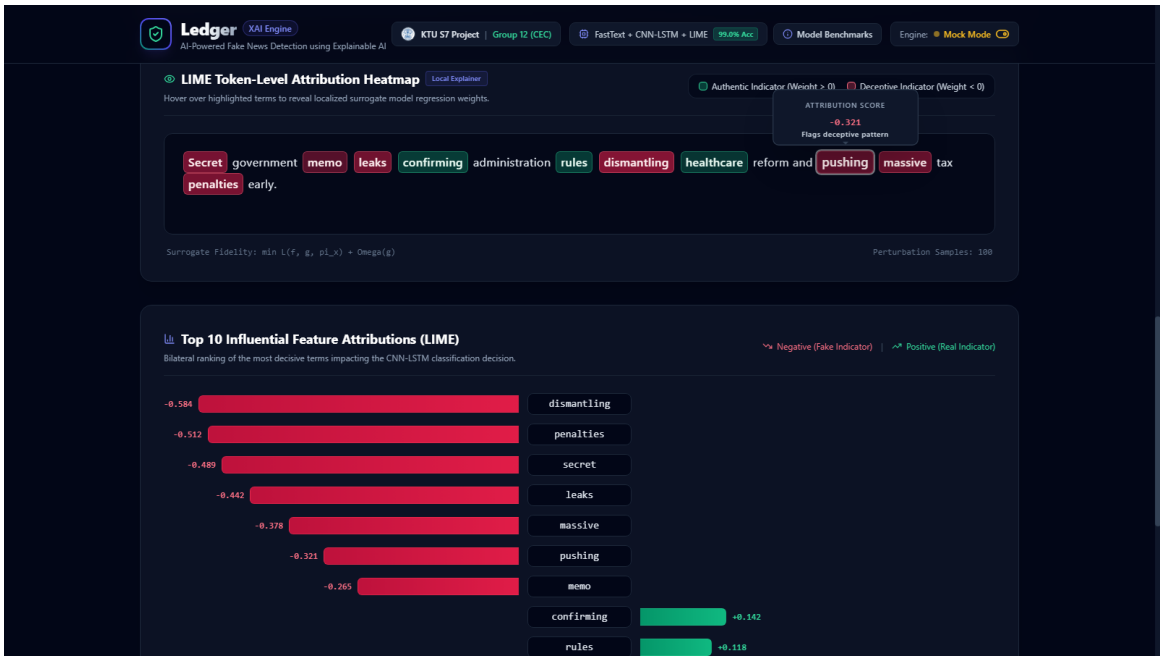


Figure 9.4: Top 10 Feature Importance Ranking and Attribution Tooltip

9.4.5 Model Specifications and Benchmark Modal

Figure 9.5 presents the in-app educational modal accessible from the navigation bar. It details target neural pipeline specifications alongside reference benchmark metrics across the WELFake (99.0 % accuracy), FakeNewsNet (97.0 % accuracy) and LIAR corpora.

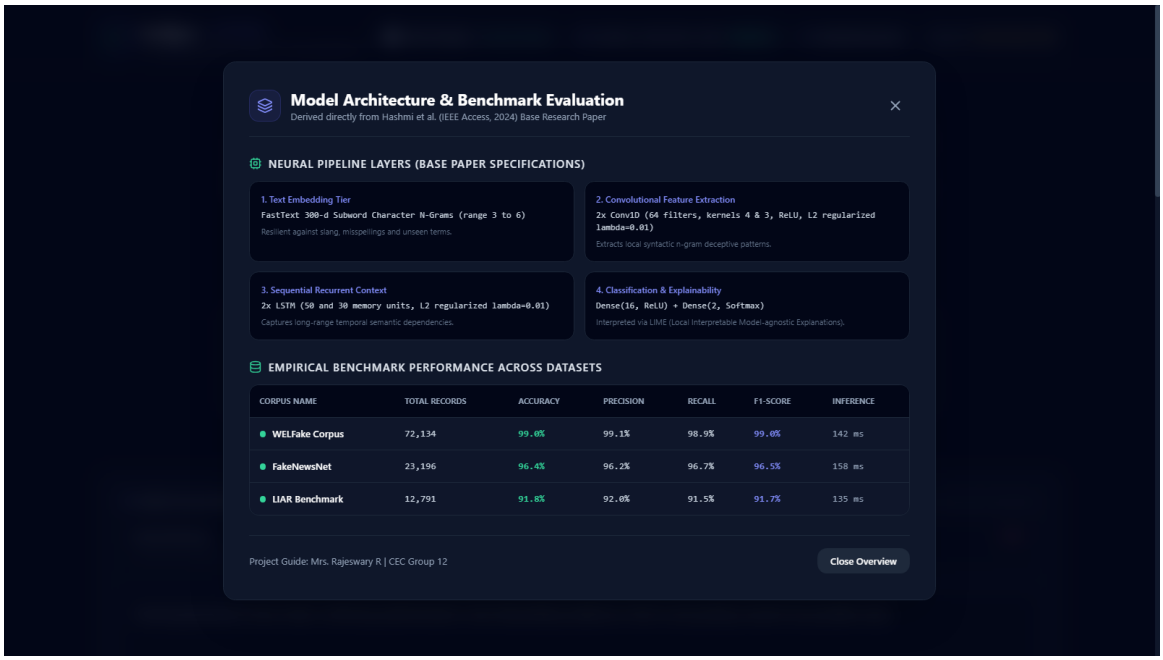


Figure 9.5: Model Architecture and Benchmark Metrics Modal

9.4.6 Mobile Device Responsiveness

Figure 9.6 demonstrates the responsive design of the prototype on a simulated smartphone display. The interface dynamically stacks statistics, preset buttons, input areas and verdict cards vertically, maintaining touch accessibility and legible attribution highlights. It is achieved by creating many CSS breakpoints which dynamically adjust or scale the size of each component so as to fit the display at almost all resolutions - whether it is tablet, smartphone, laptops, computers or any other devices of varying screen resolutions.

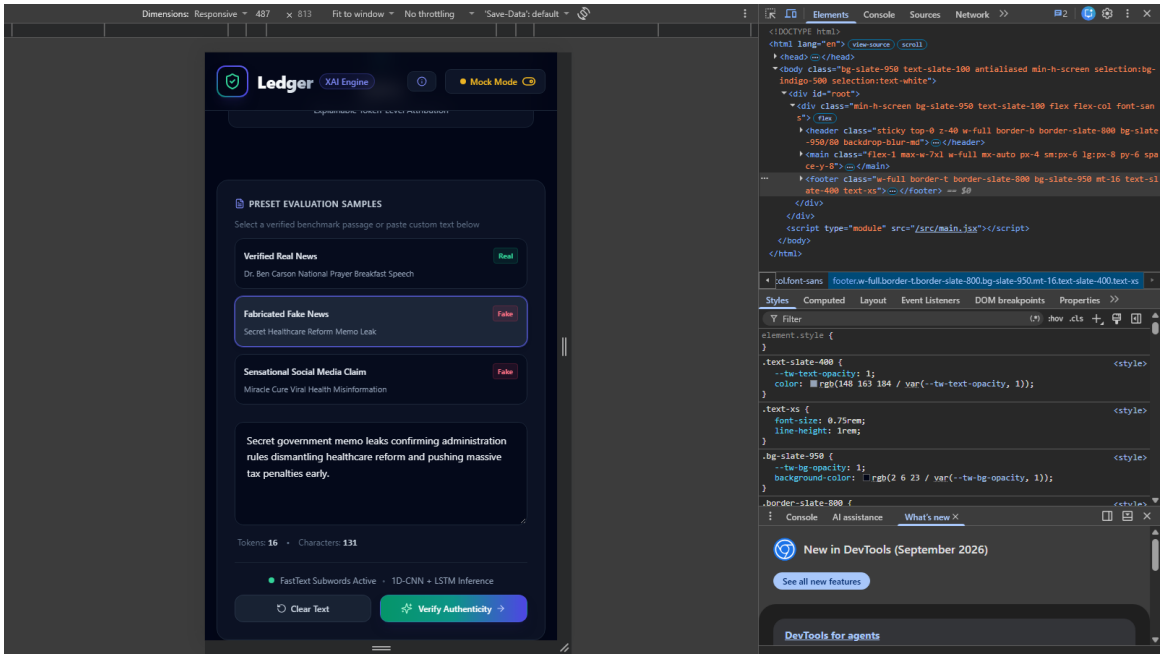


Figure 9.6: Mobile Responsive Interface Layout

9.5 Technical Setup of the Implemented Prototype

The prototype application is built on modern web technologies ensuring rapid execution and portable deployment:

- **Client Framework:** React.js (version 18) utilizing functional components, hooks for local state management and modular component architecture.
- **Build Tooling:** Vite build environment providing instant hot-module replacement (HMR) and optimized static asset compilation.
- **Styling and Typography:** Tailwind CSS with the official typography plugin for utility-first, responsive layouts and custom semantic color palettes.
- **Iconography:** Lucide React icon library for visual controls and badges.
- **Execution Environment:** Node.js 18 LTS runtime executing on commodity development hardware and served locally via `http://localhost:5173`.

The standalone architecture of the interim prototype operates without external server dependencies, enabling immediate local verification, reliable demonstrations and consistent graphical evaluation during Phase I academic assessments.

REFERENCES

1. E. Hashmi, S. Y. Yayilgan, M. M. Yamin, S. Ali and M. Abomhara, "Advancing Fake News Detection: Hybrid Deep Learning With FastText and Explainable AI," *IEEE Access*, vol. 12, pp. 44463–44481, 2024. DOI: 10.1109/ACCESS.2024.3381038.

2. P. K. Verma, P. Agrawal, I. Amorim and R. Prodan, "WELFake: Word Embedding Over Linguistic Features for Fake News Detection," *IEEE Transactions on Computational Social Systems*, vol. 8, no. 4, pp. 881–893, Aug. 2021.

3. K. Shu, D. Mahudeswaran, S. Wang, D. Lee and H. Liu, "FakeNewsNet: A Data Repository with News Content, Social Context and Spatiotemporal Information for Studying Fake News on Social Media," *Big Data*, vol. 8, no. 3, pp. 171–188, Jun. 2020.

4. M. Umer, Z. Imtiaz, M. Ahmad, M. Nappi, C. Medaglia, G. S. Choi and A. Mehmood, "Impact of Convolutional Neural Network and FastText Embedding on Text Classification," *Multimedia Tools and Applications*, vol. 82, no. 4, pp. 5569–5585, Feb. 2023.

5. R. K. Kaliyar, A. Goswami and P. Narang, "FakeBERT: Fake News Detection in Social Media using BERT and Deep Learning," *Soft Computing*, vol. 25, no. 18, pp. 11771–11788, Sep. 2021.

6. S. D. M. Kumar and A. M. Chacko, "A Systematic Survey on Explainable AI Applied to Fake News Detection," *Engineering Applications of Artificial Intelligence*, vol. 122, Art. no. 106087, Jun. 2023.

7. K. Soga, S. Yoshida and M. Muneyasu, "Exploiting Stance Similarity and Graph Neural Networks for Fake News Detection," *Pattern Recognition Letters*, vol. 177, pp. 26–32, Jan. 2024.

8. A. Altheneyan and A. Alhadlaq, "Big Data ML-Based Fake News Detection Using Distributed Learning," *IEEE Access*, vol. 11, pp. 29447–29463, 2023.

9. W. Y. Wang, "'Liar, Liar Pants on Fire': A New Benchmark Dataset for Fake News Detection," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2017,

pp. 422–426.

10. J. A. Nasir, O. S. Khan and I. Varlamis, “Fake news detection: A hybrid CNN-RNN based approach,” *Int. J. Inf. Manage. Data Insights*, vol. 1, no. 2, Art. no. 100032, Nov. 2021.

11. J. C. S. Reis, A. Correia, F. Murai, A. Veloso and F. Benevenuto, “Supervised Learning for Fake News Detection: An Explainable Approach,” *IEEE Internet Computing*, vol. 23, no. 2, pp. 46–53, Mar./Apr. 2019.